

Integrated Experiments — Problem Statement

Decision Tree Classification & Q-Learning Grid World

Course:	Artificial Intelligence (CSA17)	Course Outcome:	CO4 (BL3)
Assessment:	Tool 3 — Annexure A	Weightage:	40%
Questions:	2 (with sub-parts a/b/c)	Total Marks:	20
Duration:	45 Minutes	Domain:	Classification & Reinforcement Learning

Experiment 1 — Decision Tree Classification

Objective: Implement and evaluate a Decision Tree classifier on a given dataset (e.g., Iris / Play-Tennis) using Python or any ML library.

(a) Data Loading & Pre-processing

Load the dataset, pre-process it (handle missing values, encoding), and split it into training and testing sets. Display the first 5 rows and data types.

(b) Model Construction

Build a Decision Tree model using the Gini Index / Information Gain as the splitting criterion. Print the tree structure and identify the root node with justification.

(c) Evaluation

Evaluate the model using accuracy score, confusion matrix, and classification report. Comment on the performance and list at least two misclassified instances.

Experiment 2 — Reinforcement Learning: Q-Learning

Objective: Implement Q-Learning for a simple Grid World (4x4) environment and observe the agent's learned policy.

(a) Environment Definition

Define the environment: states, actions (Up/Down/Left/Right), and the reward structure (+10 Goal, -1 each step, -5 obstacle). Initialise the Q-table with zeros and state the hyperparameters (α , γ , ϵ).

(b) Q-Learning Training

Run the Q-Learning algorithm for at least 100 episodes. Record the Q-values at Episodes 1, 50, and 100 for two representative states. Show how the Q-table evolves.

(c) Policy Extraction & Convergence

Extract and display the final learned policy (optimal action at each state). Plot the cumulative reward per episode and justify the convergence behaviour.

Evaluation Rubric

Criteria	Weightage	Focus of Assessment
Understanding of Concepts	25%	Thorough understanding, integration of multiple concepts
Experimental Design	30%	Well-planned, systematic, optimal implementation
Use of Tools	20%	Effective, advanced, correctly configured use of tools
Analysis of Results	15%	Insightful, accurate interpretation and validation
Documentation	10%	Clear, well-structured documentation with diagrams/code/results

Deliverables Expected

- Problem Statement document (this document)
- Solution document with complete written answers to all sub-questions
- Python source code implementing both experiments
- Output visualisation(s) (decision tree, confusion matrix, grid world, policy, reward curve)
- A professional project report summarising methodology and results
- A README file describing how to run the project